

Conteúdo

1 DP

1.1	Knapsack	1
1.2	LIS	1
1.3	LCS	1
1.4	Subset-sum	1

2 Annotation

2.1	Annotation	2
-----	----------------------	---

3 Graphs

3.1	DSU	2
3.2	Dijkstra	2
3.3	Floyd-Warshall	2
3.4	Bellman-Ford	2
3.5	Kruskal	2

4 Range Queries

4.1	Merge sort	2
4.2	Count inversions (Divide and Conquer)	2
4.3	Segment Tree (range min)	2
4.4	Fenwick Tree	2
4.5	Count inversions (Fenwick + compression)	2
4.6	K-th query on frequency SegTree	2
4.7	Binary search on SegTree	2
4.8	Subarray Sum Queries	2

5 Strings

5.1	Hash	5
5.2	KMP	5
5.3	Manacher's algorithm	6
5.4	Trie	6

6 Math

6.1	Divisors	7
6.2	Kadane's algorithm	7
6.3	Modular Combinations	7
6.4	Sieve of Eratosthenes	7
6.5	Totient function	7

1 DP

1.1 Knapsack

```
vector<int> w, v;
// W = knaps capacity, n = n items
// w[i] = weight, v[i] = value
int knapsack(int n, int W) {
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int k = W - w[i]; k >= 0; k--) {
            dp[k + w[i]] = max(dp[k + w[i]], dp[k] + v[i]);
        }
    }

    return dp[W];
}
```

1.2 LIS

```
int lis(vector<int>& nums) {
    vector<int> v;

    for (auto x: nums) {
        auto it = lower_bound(v.begin(), v.end(), x);
        // lower -> a > b
        // upper -> a >= b (allow duplicates)

        if (it == v.end()) v.push_back(x);
        else *it = x;
    }

    return v.size();
}
```

1.3 LCS

```
int lcs(string a, string b) {
    vector<vector<int>>> m(a.size()+1, vector<int>(b.size()+1));
    for (int i = a.size(); i >= 0; i--) {
        for (int j = b.size(); j >= 0; j--) {
            if (i == a.size() || j == b.size()) {
                m[i][j] = 0; continue;
            }

            if (a[i] == b[j]) m[i][j] = 1 + m[i+1][j+1];
            else m[i][j] = max(m[i][j+1], m[i+1][j]);
        }
    }

    return m[0][0]; // result
}
```

1.4 Subset-sum

```
vector<int> w, v;
vector<vector<int>>> memo;

// max sum(v) with sum(w) <= remW
int subsum(int id, int remW) {
    if ((id == w.size()) || (remW == 0)) return 0;
    int &ans = memo[id][remW];
    if (ans != -1) return ans;
    if (w[id] > remW) return ans = subsum(id+1, remW);
    return ans = max(subsum(id + 1, remW),
                    v[id]+subsum(id+1, remW-w[id]));
}

// which values can be created from a list a (int)
int subsumi(int n, int max_v, vector<int> a) {
    vector<int> m(max_v + 10, 0);

    m[0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = max_v; j >= a[i]; j--) {
            m[j] |= m[j - a[i]];
        }
    }

    return m[max_v];
}
```

2 Annotation

2.1 Annotation

```
g++ "$1.cpp" -o "$1" -Wall -Wextra -O2 -Wshadow -fsanitize=address,undefined
```

3 Graphs

3.1 DSU

```
struct DSU {
    vector<int> parent, size;
    DSU(int n) : parent(n+1), size(n+1) {
        for(int i=0; i<=n; i++) {parent[i] = i; size[i] = 1;}
    }

    int find(int u) {
        if (parent[u] == u) return u;

        return parent[u] = find(parent[u]);
    }

    void join(int u, int v) {
        u = find(u); v = find(v);
        if (u == v) return;

        if (size[u] < size[v]) swap(u, v);
        parent[v] = u;
        size[u] += size[v];
    }
};
```

3.2 Dijkstra

```
vector<pii> g[MAXN];

void dijkstra(int s, int n, vector<ll>& d) {
    d.assign(n, INF);

    priority_queue<pii, vector<pii>, greater<pii>> q;
    d[s] = 0; q.push({0, s});

    while (!q.empty()) {
        auto [d_u, u] = q.top(); q.pop();
        if (d_u > d[u]) continue;

        for (auto [v, w] : g[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                q.push({d[v], v});
            }
        }
    }
}
```

3.3 Floyd-Warshall

```
bool floyd_warshall(int n, vector<vector<pii>>& g) {
    vector<vector<ll>> d(n);
    d.assign(n, vector<ll>(n, INF));
    rep(i, n) d[i][i] = 0;
    rep(i, n) for (auto [j,w]: g[i]) d[i][j] = w;

    rep(k, n)
    rep(i, n)
    rep(j, n)
        if (d[i][k] < INF && d[k][j] < INF)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    rep(i, n)
        if (d[i][i] < 0) return true; // cycle

    return false;
}
```

3.4 Bellman-Ford

```
vector<tiit> edges;

void bellman_ford(int s, int n, vector<ll>& dist) {
    dist.assign(n, INF);
    dist[s] = 0;
    vector<int> p(n, -1);

    int x = -1; // cycle detect
    for (int i = 0; i < n; i++) {
        x = -1;
        for (int j = 0; j < edges.size(); j++) {
            auto [u, v, w] = edges[j];
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                x = v; p[v] = u;
            }
        }
    }

    if (x == -1) {} // no cycle
    else {
        int y = x;
        for (int i = 0; i < n; i++)
            y = p[y];

        vector<int> cycle;
        for (int cur = y;; cur = p[cur]) {
            cycle.push_back(cur);
            if (cur == y && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end()); // negative cycle
    }
}
```

3.5 Kruskal

```
struct edge {int u, v, w};
bool comp(edge a, edge b) {return a.w < b.w;};

vector<edge> edges;

int kruskal(int n) {
    DSU ds(n);
    sort(edges.begin(), edges.end(), comp);

    int cost;
```

```

for (auto [u, v, w] : edges) {
    if (ds.find(u) != ds.find(v)) {
        cost += w;
        ds.join(u, v);
    }
}

return cost;
}

```

4 Range Queries

4.1 Merge sort

```

void merge_sort(vector<int>& a, int l, int r) {
    if (l >= r) return;

    int mid = (l + r) / 2;
    merge_sort(a, l, mid);
    merge_sort(a, mid + 1, r);

    vector<int> temp;
    temp.reserve(r - l + 1);

    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else temp.push_back(a[j++]);
    }

    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }
}

```

4.2 Count inversions (Divide and Conquer)

```

ll merge_count(vector<int>& a, int l, int r) {
    if (l >= r) return 0;

    int mid = (l + r) / 2;
    ll inv = merge_count(a, l, mid) + merge_count(a, mid + 1, r);

    vector<int> temp;
    temp.reserve(r - l + 1);

    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else {
            inv += (mid - i + 1);
            temp.push_back(a[j++]);
        }
    }

    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }
}

```

```

return inv;
}

ll count_inversions(vector<int> a) {
    if (a.empty()) return 0;
    return merge_count(a, 0, (int)a.size() - 1);
}

```

4.3 Segment Tree (range min)

Code by SamuelH12
 -> Segment Tree com:
 - Query em Range
 - Update em Ponto

```

build(1, 1, n, lista);
query(1, 1, n, a, b);
update(1, 1, n, i, x);

```

n	tamanho
[a, b]	intervalo da busca
i	posicao a ser modificada
x	novo valor da posicao i
lista	vector de elementos originais

```

Build: O(N)
Query: O(log N)
Update: O(log N)

```

```

const int MAXN = 1e6 + 5;
int seg[4*MAXN];

int query(int no, int l, int r, int a, int b){
    if(b < l || r < a) return 0;
    if(a <= l && r <= b) return seg[no];

    int m=(l+r)/2, e=no*2, d=no*2+1;

    return query(e, l, m, a, b) + query(d, m+1, r, a, b);
}

```

```

void update(int no, int l, int r, int pos, int v){
    if(pos < l || r < pos) return;
    if(l == r){seg[no] = v; return; }

    int m=(l+r)/2, e=no*2, d=no*2+1;

    update(e, l, m, pos, v);
    update(d, m+1, r, pos, v);

    seg[no] = seg[e] + seg[d];
}

void build(int no, int l, int r, vector<int> &lista){
    if(l == r){ seg[no] = lista[l]; return; }

    int m=(l+r)/2, e=no*2, d=no*2+1;

    build(e, l, m, lista);
    build(d, m+1, r, lista);

    seg[no] = seg[e] + seg[d];
}

```

// only if necessary

```

int lower_bound_prefix(int no, int l, int r, int k){
    if(seg[no] < k) return -1;    // not enough sum in this segment
    if(l == r) return l;

    int m = (l + r) / 2, e = no * 2, d = no * 2 + 1;

    if(seg[e] >= k) return lower_bound_prefix(e, l, m, k);
    return lower_bound_prefix(d, m + 1, r, k - seg[e]);
}

int lower_bound_prefix(int n /*tree size*/, int k){
    if(k <= 0) return 1;
    return lower_bound_prefix(1, 1, n, k);
}

```

4.4 Fenwick Tree

```

struct Fenwick {
    int n;
    vector<ll> bit;

    Fenwick() {}
    Fenwick(int n): n(n), bit(n + 1, 0) {}

    // add value to index idx (1-indexed)
    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += val;
        }
    }

    // sum of [1..idx]
    ll pref_sum(int idx) const {
        ll ans = 0;
        for (; idx > 0; idx -= idx & -idx) {
            ans += bit[idx];
        }
        return ans;
    }

    ll range_sum(int l, int r) const {
        if (l > r) return 0;
        return pref_sum(r) - pref_sum(l - 1);
    }
};

```

4.5 Count inversions (Fenwick + compression)

```

struct Fenwick {
    int n;
    vector<ll> bit;

    Fenwick(int n): n(n), bit(n + 1, 0) {}

    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += val;
        }
    }

    ll pref_sum(int idx) const {
        ll ans = 0;
        for (; idx > 0; idx -= idx & -idx) {
            ans += bit[idx];
        }
    }
};

```

```

    }
    return ans;
}

ll count_inversions_fenwick(vector<int> a) {
    if (a.empty()) return 0;

    vector<int> comp = a;
    sort(comp.begin(), comp.end());
    comp.erase(unique(comp.begin(), comp.end()), comp.end());

    int m = (int)comp.size();
    Fenwick fw(m);

    ll inv = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        int x = (int)(lower_bound(comp.begin(), comp.end(), a[i]) - comp.begin()) + 1;
        inv += fw.pref_sum(x - 1);
        fw.add(x, 1);
    }

    return inv;
}

```

4.6 K-th query on frequency SegTree

```

struct SegTreeFreq {
    int n;
    vector<int> st;

    SegTreeFreq(int n): n(n), st(4 * n + 5, 0) {}

    void add(int p, int l, int r, int idx, int delta) {
        if (l == r) {
            st[p] += delta;
            return;
        }

        int m = (l + r) / 2;
        if (idx <= m) add(2 * p, l, m, idx, delta);
        else add(2 * p + 1, m + 1, r, idx, delta);

        st[p] = st[2 * p] + st[2 * p + 1];
    }

    void add(int idx, int delta) {
        add(1, 0, n - 1, idx, delta);
    }

    // returns index of k-th element (1-indexed k), or -1 if k is invalid
    int kth(int p, int l, int r, int k) const {
        if (k <= 0 || k > st[p]) return -1;
        if (l == r) return l;

        int m = (l + r) / 2;
        if (st[2 * p] >= k) return kth(2 * p, l, m, k);
        return kth(2 * p + 1, m + 1, r, k - st[2 * p]);
    }

    int kth(int k) const {
        return kth(1, 0, n - 1, k);
    }
};

```

4.7 Binary search on SegTree

```

struct SegTreeMax {
    int n;
    vector<ll> st;

    SegTreeMax(const vector<ll>& a) {
        n = (int)a.size();
        st.assign(4 * n + 5, 0);
        build(1, 0, n - 1, a);
    }

    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            st[p] = a[l];
            return;
        }

        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m + 1, r, a);
        st[p] = max(st[2 * p], st[2 * p + 1]);
    }

    void update(int p, int l, int r, int idx, ll val) {
        if (l == r) {
            st[p] = val;
            return;
        }

        int m = (l + r) / 2;
        if (idx <= m) update(2 * p, l, m, idx, val);
        else update(2 * p + 1, m + 1, r, idx, val);

        st[p] = max(st[2 * p], st[2 * p + 1]);
    }

    // first index with value >= k, or -1 if it does not exist
    int first_ge(int p, int l, int r, ll k) const {
        if (st[p] < k) return -1;
        if (l == r) return l;

        int m = (l + r) / 2;
        if (st[2 * p] >= k) return first_ge(2 * p, l, m, k);
        return first_ge(2 * p + 1, m + 1, r, k);
    }

    int first_ge(ll k) const {
        return first_ge(1, 0, n - 1, k);
    }

    // consume k from first element >= k
    int consume_first_ge(vector<ll>& a, ll k) {
        int idx = first_ge(k);
        if (idx == -1) return -1;

        a[idx] -= k;
        update(1, 0, n - 1, idx, a[idx]);
        return idx;
    }
};

```

4.8 Subarray Sum Queries

```

struct Node {
    ll sum, pref, suff, best;
};

struct SegTreeMaxSubarray {
    int n;

```

```

    vector<Node> st;

    SegTreeMaxSubarray(const vector<ll>& a) {
        n = (int)a.size();
        st.assign(4 * n + 5, {0, 0, 0, 0});
        build(1, 0, n - 1, a);
    }

    Node merge_node(const Node& L, const Node& R) const {
        Node ans;
        ans.sum = L.sum + R.sum;
        ans.pref = max(L.pref, L.sum + R.pref);
        ans.suff = max(R.suff, R.sum + L.suff);
        ans.best = max({L.best, R.best, L.suff + R.pref});
        return ans;
    }

    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            st[p] = {a[l], a[l], a[l], a[l]};
            return;
        }

        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m + 1, r, a);
        st[p] = merge_node(st[2 * p], st[2 * p + 1]);
    }

    void update(int p, int l, int r, int idx, ll val) {
        if (l == r) {
            st[p] = {val, val, val, val};
            return;
        }

        int m = (l + r) / 2;
        if (idx <= m) update(2 * p, l, m, idx, val);
        else update(2 * p + 1, m + 1, r, idx, val);

        st[p] = merge_node(st[2 * p], st[2 * p + 1]);
    }

    void update(int idx, ll val) {
        update(1, 0, n - 1, idx, val);
    }

    ll max_subarray_sum() const {
        return st[1].best;
    }
};

```

5 Strings

5.1 Hash

```

String Hash
precalc()    -> O(N)
StringHash() -> O(|S|)
gethash()    -> O(1)

```

StringHash hash(s); -> Cria uma struct de StringHash para a string s
hash.gethash(l, r); -> Retorna o hash do intervalo L R da string (0-Indexado)

IMPORTANTE! Chamar precalc() no inicio do codigo

const ll MOD = 131'807'699; -> Big Prime Number

```
const ll base = 127; // Random number larger than the Alphabet
```

```
const int MAXN = 1e6 + 5;
```

```
const ll MOD = 1e9 + 7; //WA? Muda o MOD e a base
const ll base = 153;
```

```
ll expb[MAXN];
```

```
void precalc(){
    expb[0] = 1;
    for(int i=1; i<MAXN; i++)
        expb[i] = (expb[i-1]*base)%MOD;
}
```

```
struct StringHash{
    vector<ll> hsh;

    StringHash(string &s){
        hsh.assign(s.size()+1, 0);
        for(int i=0; i<s.size(); i++)
            hsh[i+1] = (hsh[i] * base % MOD + s[i]) % MOD;
    }

    ll gethash(int l, int r){
        return (MOD + hsh[r+1] - hsh[l]*expb[r-l+1] % MOD) % MOD;
    }
};
```

5.2 KMP

KMP - Find all occurrences of a pattern string inside a text string

matching(s, t) retorna os indices das ocorrencias de s em t
autKMP constroi o automato do KMP

Complexidades:
pi - O(n)
match - O(n + m)
construir o automato - O(|sigma|*n)
n = |padrao| e m = |texto|

```
template<typename T> vector<int> pi(T s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); i++) {
        while (j and s[j] != s[i]) j = p[j-1];
        if (s[j] == s[i]) j++;
        p[i] = j;
    }
    return p;
}

template<typename T> vector<int> matching(T& s, T& t) {
    vector<int> p = pi(s), match;
    for (int i = 0, j = 0; i < t.size(); i++) {
        while (j and s[j] != t[i]) j = p[j-1];
        if (s[j] == t[i]) j++;
        if (j == s.size()) match.push_back(i-j+1), j = p[j-1];
    }
    return match;
}

struct KMPaut : vector<vector<int>> {
    KMPaut() {}
    KMPaut (string& s) : vector<vector<int>>(26, vector<int>(s.size()+1)) {
        vector<int> p = pi(s);
```

```
auto& aut = *this;
aut[s[0]-'a'][0] = 1;
for (char c = 0; c < 26; c++)
    for (int i = 1; i <= s.size(); i++)
        aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
};
```

5.3 Manacher's algorithm

Manacher Algorithm
Find every palindrome in string
Complexidade: O(N)

```
vector<int> manacher(string &st) {
    string s = "$_";
    for(char c : st){ s += c; s += "_"; }
    s += "#";

    int n = s.size()-2;

    vector<int> p(n+2, 0);
    int l=1, r=1;

    for(int i=1, j; i<=n; i++)
    {
        p[i] = max(0, min(r-i, p[l+r-i])); //atualizo o valor atual para o valor do palindromo
                                           //espelho na string ou para o total que esta contido

        while( s[i-p[i]] == s[i+p[i]] ) p[i]++;

        if( i+p[i] > r ) l = i-p[i], r = i+p[i];
    }

    for(auto &x : p) x--; //o valor de p[i] e igual ao tamanho do palindromo + 1

    return p;
}
```

5.4 Trie

Trie - Arvore de Prefixos
insert(P) - O(|P|)
count(P) - O(|P|)
MAXS - Soma do tamanho de todas as Strings
sigma - Tamanho do alfabeto

```
const int MAXS = 1e5 + 10;
const int sigma = 26;

int trie[MAXS][sigma], terminal[MAXS], z = 1;

void insert(string &p){
    int cur = 0;

    for(int i=0; i<p.size(); i++){
        int id = p[i] - 'a';

        if(trie[cur][id] == -1 ){
            memset(trie[z], -1, sizeof trie[z]);
            trie[cur][id] = z++;
        }
```

```

    }

    cur = trie[cur][id];
}

terminal[cur]++;
}

int count(string &p){
    int cur = 0;

    for(int i=0; i<p.size(); i++){
        int id = (p[i] - 'a');

        if(trie[cur][id] == -1) return 0;

        cur = trie[cur][id];
    }
    return terminal[cur];
}

void init(){
    memset(trie[0], -1, sizeof trie[0]);
    z = 1;
}

```

6 Math

6.1 Divisors

get_divisors(n) -- $O(\sqrt{n})$

```

vector<int> get_divisors(int n){
    vector<int> divisors;

    for(int i = 1; i*i <= n; i++){
        if(n % i == 0){
            divisors.push_back(i);
            if(i != n/i) divisors.push_back(n/i);
        }
    }
    return divisors;
}

```

6.2 Kadane's algorithm

Algoritmo de Kadane
Consegue o max subarray sum
 $O(N)$

```

ll kadane(vector<ll> &arr) {
    ll answ = arr[0];
    ll maxEnding = arr[0];

    for (int i = 1; i < arr.size(); i++) {
        maxEnding = max(maxEnding + arr[i], arr[i]);

        answ = max(answ, maxEnding);
    }
}

```

```

return answ;
}

```

6.3 Modular Combinations

Combinacao modular
Inverso modular
Exponenciacao rapida ($O(\log P)$) - p: potencia
 $O(N)$ fatorial

```

#define MOD 9987123

vector<ll> fact(1e6, -1);

void pre(){
    fact[0] = 1;
    for(ll i = 1; i < fact.size(); i++){
        fact[i] = (fact[i-1] * i) % MOD;
    }
}

ll fexp(ll a, ll b){
    ll ans = 1;

    while(b){
        if(b & 1) ans = (ans * a) % MOD;
        a = (a*a) % MOD;
        b >>= 1;
    }

    return ans;
}

ll inv(ll a, ll p){
    return fexp(a, p - 2);
}

ll comb(ll n, ll k, ll p){
    return ((fact[n] * inv(fact[k], p)) % p) * inv(fact[n-k], p) % p;
}

```

6.4 Sieve of Eratosthenes

Sieve of Eratosthenes - Encontra o maior divisor primo
Fact -> Fatora um numero <= limite, sai ordenada
Crivo calcula a lista de primos

Crivo_mobius
- 1 se $n=1$
- 0 se n tem algum fator primo ao quadrado
- $(-1)^k$ se n e produto de k primos distintos

A funcao fact adiciona o numero 1 se vc tentar fatorar o 1.
Complexidade:
crivo - $O(n \log(\log N))$
fact - $O(\log(n))$

```

#define MAX 1000
int divi[MAX];
vector<int> primes;

```

```

void crivo(int lim) {
    divi[1] = 1;
    for (int i = 2; i <= lim; i++) {
        if (divi[i] == 0) divi[i] = i, primes.push_back(i);
        for (int j : primes) {
            if (j > divi[i] or i*j > lim) break;
            divi[i*j] = j;
        }
    }
}

void crivo_mobius(ll lim) {
    mobius[1] = 1;
    for (int i = 2; i <= lim; i++) {
        if (!divi[i]) {
            divi[i] = i;
            primes.push_back(i);
            mobius[i] = -1; // primo
        }
        for (int p : primes) {
            if (p > divi[i] || 1LL * i * p > lim) break;
            divi[i*p] = p;
            if (i % p == 0) {
                mobius[i*p] = 0; // quadrado
                break;
            } else {
                mobius[i*p] = -mobius[i];
            }
        }
    }
}

void fact(vector<int>& v, int n) {
    if (n != divi[n]) fact(v, n/divi[n]);
    v.push_back(divi[n]);
}

```

6.5 Totient function

Totiente de Euler - Conta quantos numeros de 1 ate n sao coprimos de n

Complexidade:
 $O(\sqrt{n})$

```

// Totiente
//
// O(sqrt(n))

int tot(int n) {
    int ret = n;

    for (int i = 2; i*i <= n; i++) if (n % i == 0) {
        while (n % i == 0) n /= i;
        ret -= ret / i;
    }
    if (n > 1) ret -= ret / n;

    return ret;
}

```